

Problem Plecakowy i Programowanie Dynamiczne

Odkryjmy, jak algorytmy mogą pomóc w rozwiązywaniu skomplikowanych problemów decyzyjnych.



Co to jest problem plecakowy?

Ograniczona pojemność

Masz plecak o określonej, skończonej pojemności (np. 15 kg).

Zbiór przedmiotów

Każdy przedmiot ma swoją wagę i przypisaną wartość.

Cel: Maksymalna wartość

Chcesz wybrać przedmioty, które łącznie dadzą największą wartość, nie przekraczając pojemności plecaka.



Przykład problemu plecakowego

Pojemność plecaka: 15 kg

Dostępne przedmioty z ich wagą i wartością:

- Lampa stołowa: 2 kg, 40 zł
- Podręcznik Cormena: 2 kg, 160 zł
- Blender: 3 kg, 70 zł
- Koń na biegunach: 15 kg, 300 zł
- Kilogram tytanu: 1 kg, 70 zł
- Dziwna ozdoba: 4 kg, 25 zł
- Połamane krzesło: 5 kg, 25 zł
- Odkurzacz: 6 kg, 180 zł



Optymalny wybór: lampa, podręcznik, blender, tytan, odkurzacz.

Łączna wartość: 520 zł, Łączna waga: 14 kg

Dlaczego problem jest trudny?

1

Naiwne strategie zawodzą

Wybór najdroższych, najlżejszych lub o najlepszym stosunku wartości do wagi nie zawsze prowadzi do optimum.

2

Przykład paradoksu

Plecak 4 kg, przedmioty: (3kg, 70zł), (2kg, 40zł), (2kg, 40zł). Optymalnie: dwa przedmioty po 2 kg (80zł), a nie jeden 3kg (70zł).

3

Złożoność wykładnicza

Przeszukanie wszystkich możliwych podzbiorów przedmiotów (tzw. "brute force") ma złożoność 2^n , co jest niepraktyczne dla dużej liczby przedmiotów.



Programowanie dynamiczne – idea

Programowanie dynamiczne to potężna technika rozwiązywania problemów optymalizacyjnych, charakteryzująca się rozbijaniem dużego problemu na mniejsze, łatwiejsze do zarządzania podproblemy.

Dzielenie na podproblemy

Problem jest dzielony na mniejsze, nakładające się podproblemy.

Rozwiązanie etapami

Rozwiązujemy problemy od najprostszych do najbardziej złożonych, aż do uzyskania rozwiązania globalnego.



Definiowanie funkcji

Tworzymy funkcję $P[i][w]$, która reprezentuje maksymalną wartość dla 'i' przedmiotów i pojemności 'w'.

Zapamiętywanie wyników

Wyniki podproblemów są przechowywane, aby uniknąć ponownego obliczania.

Rekurencja i warunki brzegowe

Kluczem do programowania dynamicznego jest zdefiniowanie relacji rekurencyjnej, która pozwala na budowanie rozwiązania z wcześniej obliczonych wartości.

Warunki początkowe

Jeśli nie ma przedmiotów ($i=0$) lub pojemności ($w=0$), maksymalna wartość wynosi 0.

$$P[i][w] = 0$$

Przedmiot za duży

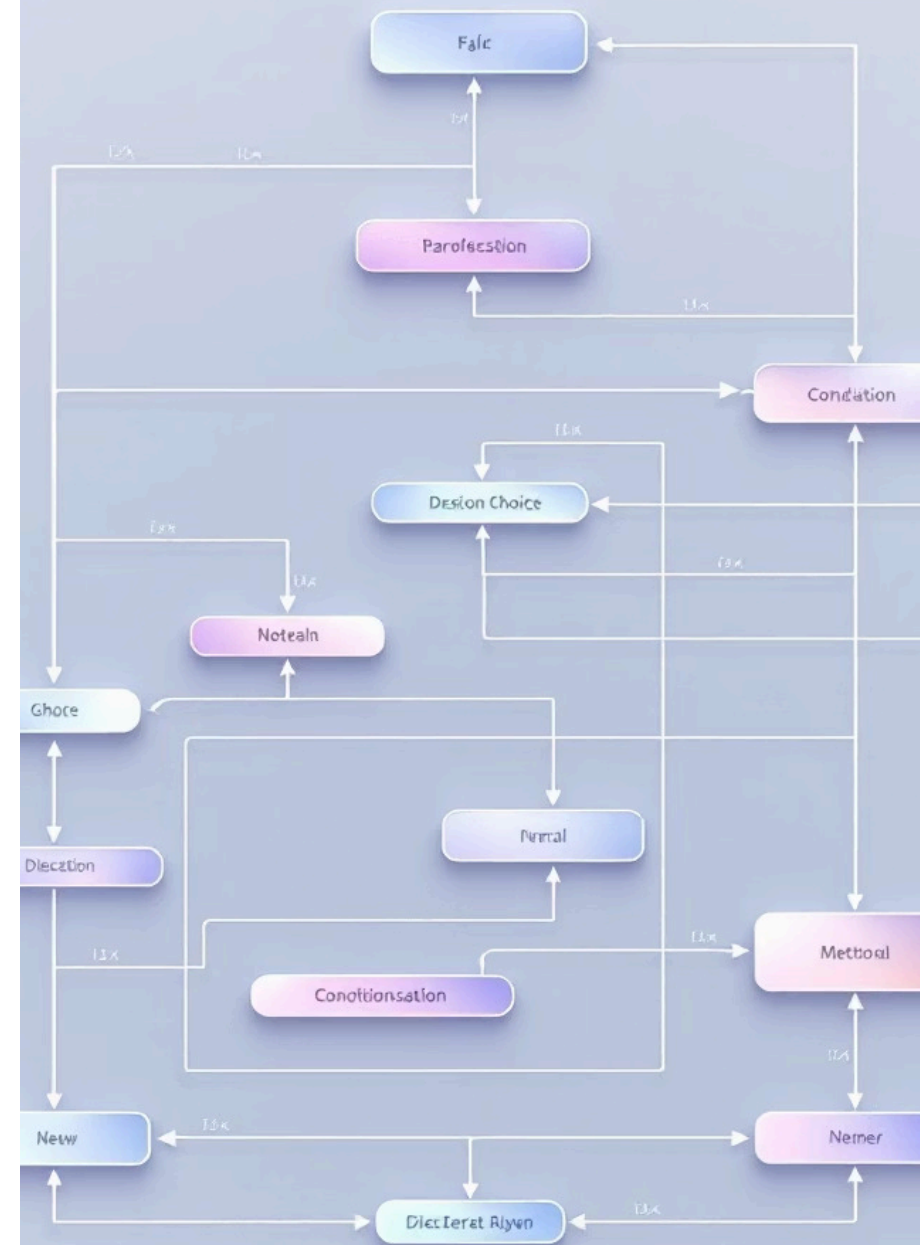
Jeśli waga i -tego przedmiotu jest większa niż aktualna pojemność plecaka ' w ', nie możemy go zabrać. Wartość pozostaje taka sama jak bez tego przedmiotu.

$$P[i][w] = P[i-1][w]$$

Możliwy wybór

Wybieramy większą z dwóch opcji: nie zabieramy i -tego przedmiotu LUB zabieramy go i dodajemy jego wartość do rozwiązania z pozostałą pojemnością.

$$P[i][w] = \max(P[i-1][w], P[i-1][w - w[i]] + v[i])$$



Pseudokod algorytmu programowania dynamicznego

```
function knapsack(weights, values, capacity):
    n = length(weights)
    // Tworzenie tabeli DP do przechowywania wyników podproblemów
    // P[i][w] = maksymalna wartość dla i przedmiotów i pojemności w
    DP = array[n + 1][capacity + 1]

    // Warunki brzegowe: jeśli brak przedmiotów lub pojemności, wartość = 0
    for i from 0 to n:
        DP[i][0] = 0
    for w from 0 to capacity:
        DP[0][w] = 0

    // Wypełnianie tabeli DP
    for i from 1 to n:
        for w from 1 to capacity:
            currentWeight = weights[i-1]
            currentValue = values[i-1]

            // Jeśli aktualny przedmiot mieści się w plecaku
            if currentWeight <= w:
                // Wybieramy max z:
                // 1. Nie bierzemy aktualnego przedmiotu (DP[i-1][w])
                // 2. Bierzemy aktualny przedmiot (currentValue + DP[i-1][w - currentWeight])
                DP[i][w] = max(DP[i-1][w], currentValue + DP[i-1][w - currentWeight])
            else:
                // Jeśli aktualny przedmiot jest za ciężki, nie bierzemy go
                DP[i][w] = DP[i-1][w]

    // Ostatecznym wynikiem jest wartość w prawym dolnym rogu tabeli DP
    return DP[n][capacity]
```